University of Stirling
Computing Science and Mathematics
CSCU9B4 - NoSQL Databases
Student ID: 30065980

Airbnb MongoDB Dataset

1 Exploring customer reviews

To answer this question, I used the fields *property_type* and *review_scores.review_scores_cleanliness*, because the task requires the average cleanliness score for each property type. An aggregation pipeline is appropriate here because the query must first remove invalid records, then group listings by *property_type*, calculate the average cleanliness score for each group, filter to keep only averages above 9.5, sort the results in descending order, and present them in a clear output format.

```
1. db.listingsAndReviews.aggregate([
2.     { $match: { property_type: { $type: "string", $ne: "" }, "review_scores.review_scores_cleanliness": { $type: "number" } }},
3.     { $group: { _id: "$property_type", averageCleanlinessScore: { $avg: "$review_scores.review_scores_cleanliness" } } },
4.     { $match: { averageCleanlinessScore: { $gt: 9.5 } } },
5.     { $sort: { averageCleanlinessScore: -1, _id: 1 } },
6.     { $project: { _id: 0, property_type: "$_id", averageCleanlinessScore: { $round: ["$averageCleanlinessScore", 2] } } }
7. ])
```

The first stage uses *$match* to keep only documents with a valid non-empty *property_type* and a numeric cleanliness score. This improves data quality before aggregation and avoids grouping records that would produce misleading results. The second stage uses *$group* to collect listings by *property_type* and calculate the mean cleanliness score with *$avg*. After the averages have been computed, a second $match is applied to keep only property types with an average cleanliness score greater than *9.5*. This stage must come after *$group* because the threshold is applied to the calculated average rather than to the original documents. The *$sort* stage then orders the output by *averageCleanlinessScore* in descending order, with *_id* in ascending order as a stable tiebreaker. Finally, *$project* removes the default grouped *_id*, renames it as *property_type*, and rounds the average score to two decimal places to make the output clearer and easier to read.

```
1.   { property_type: 'Camper/RV', averageCleanlinessScore: 10 },
 2.   { property_type: 'Earth house', averageCleanlinessScore: 10 },
 3.   { property_type: 'Houseboat', averageCleanlinessScore: 10 },
 4.   { property_type: 'Hut', averageCleanlinessScore: 10 },
 5.   { property_type: 'Train', averageCleanlinessScore: 10 },
 6.   { property_type: 'Treehouse', averageCleanlinessScore: 10 },
 7.   { property_type: 'Farm stay', averageCleanlinessScore: 9.71 },
 8.   { property_type: 'Guest suite', averageCleanlinessScore: 9.65 },
 9.   { property_type: 'Cottage', averageCleanlinessScore: 9.59 }
```

The query returned nine property types with an average cleanliness score above *9.5*. *Camper/RV*, *Earth house*, *Houseboat*, *Hut*, *Train*, and *Treehouse* each achieved a perfect average cleanliness score of *10*. The remaining qualifying categories were *Farm stay* (*9.71*), *Guest suite* (*9.65*), and *Cottage* (*9.59*). Overall, the results show that exceptionally high cleanliness ratings were concentrated in a relatively small number of property types.

2 Your Idea Holiday Destinations

2.1 Application of Analytical Hierarchy Process

The goal of *myScore* is not merely to find the listing with the highest numerical score, but to identify the best overall match for my personal user's holiday preferences. To answer this question, I designed a personalised scoring model called *myScore* to rank listings according to my holiday preferences. This means approaching the question using a multi-criteria decision-making model. The main criteria weights for *myScore* were derived using the Analytic Hierarchy Process (AHP). Saaty describes AHP as a theory of measurement that produces ratio-scale priorities from judgment, which is especially useful when making decisions that involve both tangible and intangible factors [1]. Pairwise comparisons were performed across the four top-level criteria: Confidence (CF), Review Quality (RQ), Comfort & Amenities (CA), and Affordability (AF) [1]. This model prioritises trust and evidence most strongly, then review-based quality, while comfort and affordability act as secondary supporting dimensions.

	CF	RQ	CA	AF
CF	1	3	5	7
RQ	1/3	1	3	3
CA	1/5	1/3	1	3
AF	1/7	1/5	1/3	1

Table 1 Decision Matrix (1, 3, 5, 7 Equal to Extreme Importance) + Reciprocal values

Category	Priority	Rank	(+) upper bound	(-) lower bound
CF	56.5%	1	15.9	15.9
RQ	26.2%	2	6.5%	6.5%
CA	11.8%	3	3.2%	3.2%
AF	5.5%	4	1.7%	1.7%
Consistency Ratio CR = 4.3% (CI/RI) Number of comparisons = 6 (reflects 4x4 matrix size)				

Table 2 Normalize matrix, ranked, and converted to percentage

The resulting priority vector, obtained from an online AHP calculator, assigned the greatest importance to CF (56.5%), followed by RQ (26.2%), CA (11.8%), and AF (5.5%) [2]. In AHP, the principal eigenvalue is used to assess consistency [1]. Because a perfectly consistent 4×4 matrix would have a maximum eigenvalue of 4, the obtained value of 4.117 suggests only a small deviation from perfect consistency, which leads to a low Consistency Ratio of 4.3% below the standard acceptance threshold of 10% [1]. This suggests that the judgments are tolerable and that the criteria's weighting can be used as a scoring model. The upper and lower bounds of each criterion are stable, not leaning towards any side. Adding or removing property listings from the database is unlikely to cause a rank reversal (i.e., break the ranking system).

Finally, the AHP hierarchy was designed and visualized in the draw.io diagram. Rather than relying on a single factor, myScore is structured as a multi-criteria model that balances trust, quality, comfort, and cost. The diagram shows how these criteria are organized and how the top alternatives are evaluated within that structure. The first three sample results are included to illustrate how the framework is applied in practice.

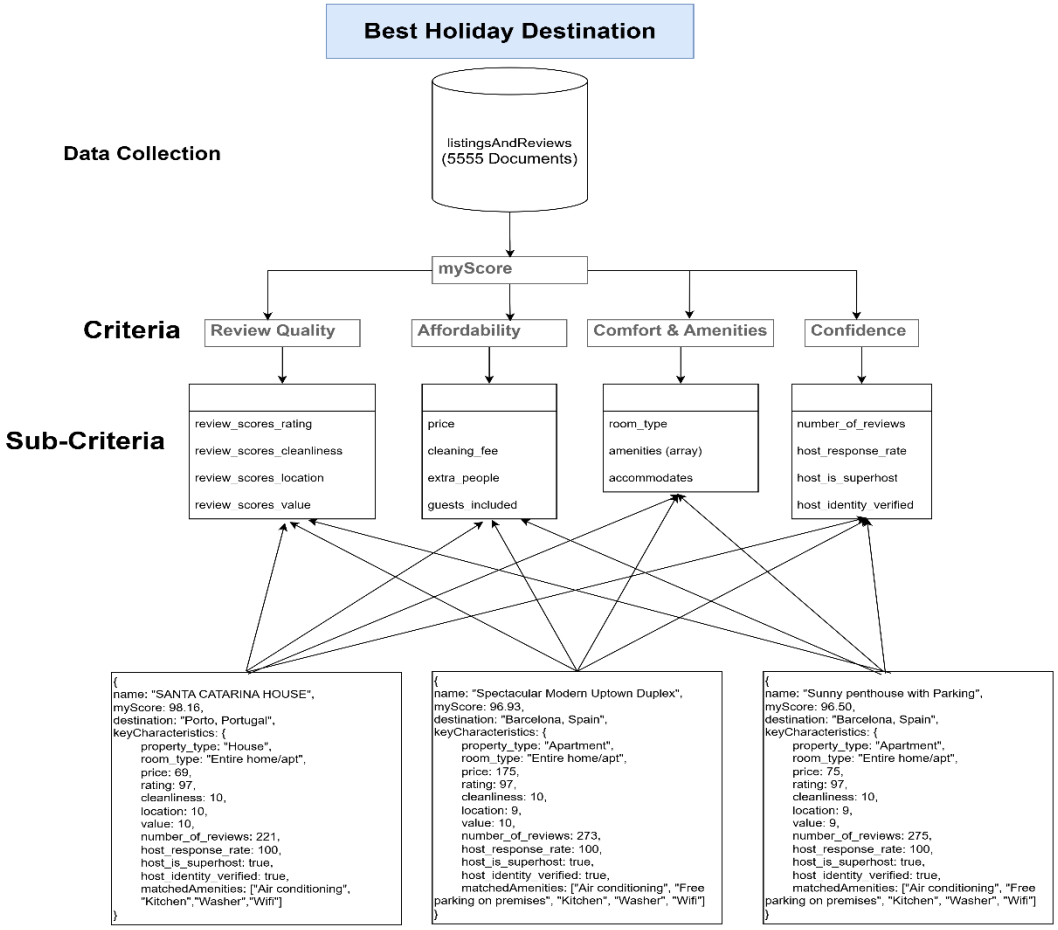


Figure 1 AHP Visualization of myScore (sampled 3 results)

Finally, we defined the mathematical calculation for *myScore* as a weighted sum model. Each sub-criterion field was normalized into a 0-1 scale. The normalized fields are averaged and grouped under the top four criteria before being multiplied by their respective AHP weights and summed to generate the final score. This approach is justified because averaging controls the internal balance within each criterion, while the AHP multiplication reflects the relative importance of each criterion in the final decision model. The formula is presented:

$$myScore = 100 \times (0.565 \cdot CF + 0.262 \cdot RQ + 0.118 \cdot CA + 0.055 \cdot AF)$$

2.2 Analytical Model Implementation

```
{ $match: {
    name: { $type: "string", $ne: "" },
    room_type: { $type: "string", $ne: "" },
    amenities: { $exists: true, $ne: [] },
    price: { $exists: true, $ne: null },
    "address.market": { $type: "string", $ne: "" },
    "address.country": { $type: "string", $ne: "" },
    "review_scores.review_scores_rating": { $type: "number" },
    "review_scores.review_scores_cleanliness": { $type: "number" },
    "review_scores.review_scores_location": { $type: "number" },
    "review_scores.review_scores_value": { $type: "number" },
    number_of_reviews: { $exists: true },
    "host.host_response_rate": { $exists: true },
    "host.host_is_superhost": { $exists: true },
    "host.host_identity_verified": { $exists: true }
} }
```

The first stage filters the collection so that only documents with the fields needed for scoring are processed. This is important because the ranking model depends on valid review, host, location, and amenity data. Applying *$match* first also improves efficiency by reducing the number of documents that later stages must transform into the aggregation. By rigorously validating data integrity using *$type*, the stage enforces business correctness and consistency. The filter stage serves as a gatekeeper to ensure the data is fit for purpose before any analytical transformation occurs.

```
{ $addFields: {
    priceValue: { $convert: { input: "$price", to: "double", onError: 0, onNull: 0 } },
    cleaningFeeValue: { $convert: { input: "$cleaning_fee", to: "double", onError: 0, onNull: 0 } },
    extraPeopleValue: { $convert: { input: "$extra_people", to: "double", onError: 0, onNull: 0 } },
    guestsIncludedValue: { $convert: { input: "$guests_included", to: "double", onError: 0, onNull: 0 } },
    responseRateValue: { $convert: { input: "$host.host_response_rate", to: "double", onError: 0, onNull: 0 } },
    bathroomsValue: { $convert: { input: "$bathrooms", to: "double", onError: 0, onNull: 0 } }
  } }
```

This next stage standardizes the raw numeric inputs before any scoring takes place. *$addFields* is used to create new working variables — *priceValue*, *cleaningFeeValue*, *extraPeopleValue*, *guestsIncludedValue*, *responseRateValue*, and *bathroomsValue* — by converting the original fields to double. This is necessary because values such as *price*, *cleaning_fee*, and bathrooms may not all be stored in a directly usable numeric form, yet later stages require consistent arithmetic operations for normalization and weighted scoring. The *$convert* operator ensures that these fields are transformed into a common numeric type, while *onError: 0* and *onNull: 0* provide a controlled fallback for invalid or missing values.

```
{ $addFields: {
    ratingScore: { $divide: ["$review_scores.review_scores_rating", 100] },
    cleanlinessScore: { $divide: ["$review_scores.review_scores_cleanliness", 10] },
    locationScore: { $divide: ["$review_scores.review_scores_location", 10] },
    valueScore: { $divide: ["$review_scores.review_scores_value", 10] },

    priceScore: { $let: { vars: { cappedPrice: { $min: ["$priceValue", 300] } },
                          in: { $divide: [{ $subtract: [300, "$$cappedPrice"] }, 300] } } },

    cleaningFeeScore: { $let: { vars: { cappedFee: { $min: ["$cleaningFeeValue", 150] } },
                                in: { $divide: [{ $subtract: [150, "$$cappedFee"] }, 150] } } },

    extraPeopleScore: { $let: { vars: { cappedExtra: { $min: ["$extraPeopleValue", 50] } },
                                in: { $divide: [{ $subtract: [50, "$$cappedExtra"] }, 50] } } },

    guestsIncludedScore: { $min: [1, { $divide: ["$guestsIncludedValue", 4] }] },

    roomTypeScore: { $switch: { branches: [ { case: { $eq: ["$room_type", "Entire home/apt"] }, then: 1 },
                                { case: { $eq: ["$room_type", "Hotel room"] }, then: 0.7 },
                                { case: { $eq: ["$room_type", "Private room"] }, then: 0.4 },
                                { case: { $eq: ["$room_type", "Shared room"] }, then: 0.1 } ],
```

```
                                        default: 0.3 } },

    amenityScore: { $divide: [ { $size: { $setIntersection:
                  [ "$amenities", ["Wifi", "Kitchen", "Air conditioning", "Washer", "Free parking on premises"] ] } }, 5 ] },

    spaceScore: { $avg: [ { $min: [1, { $divide: [{ $ifNull: ["$accommodates", 0] }, 6] }] },
                          { $min: [1, { $divide: [{ $ifNull: ["$bedrooms", 0] }, 3] }] },
                          { $min: [1, { $divide: ["$bathroomsValue", 2] }] } ] },

    reviewCountScore: { $min: [ 1, { $divide: [ { $ln: { $add: [{ $ifNull: ["$number_of_reviews", 0] }, 1] } },
                                        { $ln: 201 } ] } ] },

    responseRateScore: { $divide: ["$responseRateValue", 100] },
    superhostScore: { $cond: [{ $eq: ["$host.host_is_superhost", true] }, 1, 0] },
    identityVerifiedScore: { $cond: [{ $eq: ["$host.host_identity_verified", true] }, 1, 0] }
} }
```

Next, feature engineering and normalization are performed for the scoring model. This stage uses a mixed normalization approach to map all selected features onto a common 0–1 desirability scale using *$addFields*.

Review fields and response rate are normalized using direct scaling because their maximum values are known. *ratingScore* and *responseRateScore* are divided by *100*, while cleanliness, location, and value are divided by *10*. Cost-related variables such as *price*, *cleaning_fee*, and *extra_people* are first capped using the *$let* operator to prevent extreme values from distorting the model; for example, the price cap is set at *300* as a personal budget ceiling. Capping is appropriate here because it prevents affordability from becoming a hard filter while still penalising very expensive listings. Properties above the cap are still included in the ranking, but after capping, they receive the minimum affordability score for that component rather than being excluded entirely. After that, *$subtract* reverses the direction of the scale so that lower costs receive higher normalized values, and $divide rescales the result to the 0–1 range. The same approach is then applied to *priceScore*, *cleaningFeeScore*, and *extraPeopleScore*, using different upper bounds for each variable.

The remaining variables are also converted into bounded desirability scores, but with different methods depending on the nature of the field. *guestsIncludedScore* is normalized by dividing *guests_included* by 4, then capped at 1 using *$min*, so that listings covering more guests receive a higher score without over-dominating the model. *roomTypeScore* is handled with a *$switch*, which maps each room type to an ordered preference scale: Entire home/apt receives the highest score, followed by hotel room, Private room, and Shared room, while other types receive a default score of 0.3.

For *amenityScore*, I defined my preferred amenities as Wi-Fi, kitchen, air conditioning, washer, and free parking. *$setIntersection* is used to identify the overlap between the listing's amenities array and this preference set, and the result is normalized by dividing the count by 5. *spaceScore* summarizes suitability and capacity by averaging normalized versions of accommodations, bedrooms, and bathrooms. Each of these is scaled to a reasonable upper bound and capped at 1, ensuring that larger homes receive some benefit, but not enough to overwhelm the rest of the model.

The final part of this stage normalizes the confidence-related features. *reviewCountScore* uses a logarithmic transformation to introduce diminishing returns, so that the gain from additional reviews becomes progressively smaller at higher review volumes. *$ifNull* and *$add* are used to default missing review counts to 0 and then add 1, ensuring that $ln(1) = 0$, so a property with no reviews begins with a confidence score of 0 for this component. The score is then normalized using $ln(201)$ as the divisor, which effectively caps the contribution at 200 reviews. This prevents extremely large review counts from dominating the model while still rewarding listings with substantial review histories. Finally, *responseRateScore* is obtained by dividing the response rate by 100, while *superhostScore* and *identityVerifiedScore* are converted into binary scores using *$cond*, where true is scored as 1 and false as 0.

```
{ $addFields: {
    confidenceScore: { $avg: ["$reviewCountScore", "$responseRateScore", "$superhostScore", "$identityVerifiedScore"] },
    reviewQualityScore: { $avg: ["$ratingScore", "$cleanlinessScore", "$locationScore", "$valueScore"] },
    comfortAmenitiesScore: { $avg: ["$roomTypeScore", "$amenityScore", "$spaceScore"] },
    affordabilityScore: { $avg: ["$priceScore", "$cleaningFeeScore", "$extraPeopleScore", "$guestsIncludedScore"] }
} }
```

This stage aggregates the previously normalized field-level scores into the four main criteria used in the AHP model. Using *$addFields*, it computes *confidenceScore*, *reviewQualityScore*, *comfortAmenitiesScore*, and *affordabilityScore* as composite category scores. Each is calculated with $avg, which takes the mean of the relevant normalized sub-criterion. The individual fields within each category have already been converted to a common 0–1 scale, so they can now be combined fairly without unit bias.

```
{ $addFields: {
    myScore: { $round: [ { $multiply: [ 100, { $add: [
                                                { $multiply: [0.565, "$confidenceScore"] },
                                                { $multiply: [0.262, "$reviewQualityScore"] },
                                                { $multiply: [0.118, "$comfortAmenitiesScore"] },
                                                { $multiply: [0.055, "$affordabilityScore"] } ] } ] }, 2 ] }
} }
```

Using *$addFields* to create *myScore*, the pipeline combines the predefined sub-criterion and sums them up using *$add* after having multiplied each by the respective weights in the AHP priority vector. The weighted added value is then multiplied by 100 to represent the percentage scale and rounded to 2 decimal places using the *$round* operator.

```
{ $sort: { myScore: -1, confidenceScore: -1, reviewQualityScore: -1, priceValue: 1, _id: 1 } }
{ $limit: 5 }
```

These stages finalize the ranking. *$sort* orders listings by *myScore* in descending order, while *confidenceScore*, *reviewQualityScore*, *priceValue*, and *_id* are used as tiebreakers to keep the ranking meaningful and stable. *$limit*: 5 then returns only the top five results required by the question.

```
{ $project: {
    _id: 0,
    name: 1,
    destination: { $concat: ["$address.market", ", ", "$address.country"] },
    keyCharacteristics: {
        property_type: "$property_type",
        room_type: "$room_type",
        price: { $round: ["$priceValue", 2] },
        rating: "$review_scores.review_scores_rating",
        cleanliness: "$review_scores.review_scores_cleanliness",
        location: "$review_scores.review_scores_location",
        value: "$review_scores.review_scores_value",
        number_of_reviews: "$number_of_reviews",
        host_response_rate: "$responseRateValue",
        host_is_superhost: "$host.host_is_superhost",
        host_identity_verified: "$host.host_identity_verified",
        matchedAmenities:
       {$setIntersection: [amenities", ["Wifi", "Kitchen", "Air conditioning", "Washer", "Free parking on premises"]]} },
     myScore: 1
} }
```

This final stage formats the result to contain only the information required by the question, in a clear and readable form. *$project* is used to suppress *_id*, include the listing name, construct a human-readable destination by concatenating *address.market* and *address.country*, and then group the most relevant attributes into a single *keyCharacteristics* object. The relevancy of attributes was chosen based on what is expected to be presented on an Airbnb post. These characteristics include the property and room type, rounded price, key review measures, review count, host reliability indicators, and the subset of preferred amenities matched by the listing. The *$setIntersection* operator is particularly useful here because it shows only the amenities that overlap with the predefined preference set, rather than returning the full amenities array. Finally, *myScore* is retained as the ranking field.

```
{
  name: "SANTA CATARINA HOUSE",
  myScore: 98.16,
  destination: "Porto, Portugal",
  keyCharacteristics: {
    property_type: "House",
    room_type: "Entire home/apt",
    price: 69,
    rating: 97,
    cleanliness: 10,
    location: 10,
    value: 10,
    number_of_reviews: 221,
    host_response_rate: 100,
    host_is_superhost: true,
    host_identity_verified: true,
    matchedAmenities: ["Air conditioning", "Kitchen", "Washer", "Wifi"] }
}
{
  name: "Spectacular Modern Uptown Duplex",
  myScore: 96.93,
  destination: "Barcelona, Spain",
```

```
  keyCharacteristics: {
    property_type: "Apartment",
    room_type: "Entire home/apt",
    price: 175,
    rating: 97,
    cleanliness: 10,
    location: 9,
    value: 10,
    number_of_reviews: 273,
    host_response_rate: 100,
    host_is_superhost: true,
    host_identity_verified: true,
    matchedAmenities: ["Air conditioning", "Free parking on premises", "Kitchen", "Washer", "Wifi"]}
}
{
  name: "Sunny penthouse with Parking",
  myScore: 96.50,
  destination: "Barcelona, Spain",
  keyCharacteristics: {
    property_type: "Apartment",
    room_type: "Entire home/apt",
    price: 75,
    rating: 97,
    cleanliness: 10,
    location: 9,
    value: 9,
    number_of_reviews: 275,
    host_response_rate: 100,
    host_is_superhost: true,
    host_identity_verified: true,
    matchedAmenities: ["Air conditioning", "Free parking on premises", "Kitchen", "Washer", "Wifi"]}
}
{
  name: "At The Heart Of The Old Sultanahmet",
  myScore: 96.21,
  destination: "Istanbul, Turkey",
  keyCharacteristics: {
    property_type: "Apartment",
    room_type: "Entire home/apt",
    price: 749,
    rating: 98,
    cleanliness: 10,
    location: 10,
    value: 10,
    number_of_reviews: 151,
    host_response_rate: 100,
    host_is_superhost: true,
    host_identity_verified: true,
    matchedAmenities: ["Air conditioning", "Free parking on premises", "Kitchen", "Washer", "Wifi"]}
}
{
  name: "5*Super host Apartment+terrace 8 min from Ramblas",
  myScore: 95.99,
  destination: "Barcelona, Spain",
  keyCharacteristics: {
    property_type: "Apartment",
    room_type: "Entire home/apt",
    price: 63,
    rating: 94,
    cleanliness: 9,
    location: 10,
    value: 10,
    number_of_reviews: 320,
    host_response_rate: 100,
    host_is_superhost: true,
    host_identity_verified: true,
    matchedAmenities: ["Air conditioning", "Free parking on premises", "Kitchen", "Washer", "Wifi"]}
}
```


The results show that *myScore* is working as a balanced decision model rather than a simple price or rating sort. The top-ranked listings all have very strong confidence signals, including high review counts, perfect host response rates, super host status, and identity verification, which is consistent with the AHP weighting. They also perform strongly on review quality and amenities. This explains why a high-priced listing such as the Istanbul property can still rank highly: the model values trust and quality more than affordability alone. Overall, the top five results represent listings that are strong across multiple dimensions, not just one.

3 Modelling and scalability

3.1 Problem Understanding and Implementation Plan

Actor	CRUD Operations	Data involved in the operation	Operation type	Frequency / Latency	Retention period
Traveller	Read filtered listings	`name`, `address.market`, `address.country`, `price`, `room_type`, `property_type`, `amenities`, `review_scores.review_scores_rating`, `review_scores.review_scores_cleanliness`, `review_scores.review_scores_location`, `review_scores.review_scores_value` (Listing search fields)	Operational read	Very high/low latency critical	Long-term
Traveller	Read one listing detail	`_id`, `name`, `summary`, `description`, `property_type`, `room_type`, `bed_type`, `price`, `cleaning_fee`, `extra_people`, `guests_included`, `accommodates`, `bedrooms`, `beds`, `bathrooms`, `amenities`, `host.*`, `address.*`, `review_scores.*`, `number_of_reviews`, `first_review`, `last_review` (Listing detail fields)	Operational read	High/low latency critical	Long-term
Traveller	Read reviews for one listing	`reviews._id`, `reviews.date`, `reviews.listing_id`, `reviews.reviewer_id`, `reviews.reviewer_name`, `reviews.comments` (Reviews fields)	Operational read	Medium/medium latency	Long-term
Guest	Create a new review	`reviews._id`, `reviews.date`, `reviews.listing_id`, `reviews.reviewer_id`, `reviews.reviewer_name`, `reviews.comments` (Reviews fields 2)	Operational write	Medium/near real-time	Long-term
Host	Update listing details	`price`, `cleaning_fee`, `extra_people`, `guests_included`, `amenities`, `room_type`, `property_type`, `accommodates`, `bedrooms`, `beds`, `bathrooms`, `availability.availability_30`, `availability.availability_60`, `availability.availability_90`, `availability.availability_365` (Listings update fields)	Operational write	Medium/medium latency	Long-term
Data analyst	Compare prices across regions and property types	`address.market`, `address.country`, `property_type`, `room_type`, `price`, `cleaning_fee`, `extra_people`, `guests_included`, `first_review`, `last_review` (Price analysis fields)	Analytical read	Low to medium/batch acceptable	Medium to long-term
Admin / Moderator	Delete review or listing	Review the fields above, or the full listing document (Moderation Target Fields)	Operational write	Low / not latency critical	Policy-based

Figure 2 Use Case Definition

First, I identified the main CRUD operations performed by different actors in the system, including travellers, guests, hosts, analysts, and administrators. I then mapped the predicted fields involved in each operation and grouped them into workload categories such as listing search fields, listing detail fields, review fields, listing update fields, price analysis fields, and moderation target fields. This makes it easier to see which data is commonly accessed together, which operations are latency-sensitive, and which are operational rather than analytical. The workload is primarily read-heavy: filtered listing searches and listing-detail views occur more frequently and are more latency-sensitive than writes such as review submission, listing updates, or moderation.

The chosen redesign is the Reference Pattern. The current schema stores all review in an embedded reviews array, which becomes an unbounded-array problem when a listing accumulates several thousand reviews. This problem can overflow the MongoDB 16MB limits per document, listing documents become increasingly large, simple reads become heavier, and writes become more expensive because each new review enlarges the same parent record[3]. MongoDB identifies unbounded arrays as a schema anti-pattern and recommends references when the many sides can grow indefinitely [3]. Therefore, the most appropriate redesign is to move the detailed reviews into a separate reviews collection. Where the detail page benefits from immediate access to only a few recent reviews, the Subset Pattern can also be applied by embedding a very small recentReviews array while leaving the full review history referenced. This hybrid design is stronger than a simple normalization approach because it aligns the schema directly with the workload: frequent search and listing-detail reads remain fast, summary review data stays local to the parent document, and detailed reviews can grow independently without bloating the main listing.

Because the task only requires the review data to be remodeled, the remaining objects and arrays were left unchanged. This keeps the redesign focused on the part of the schema that creates the main scalability issue, while preserving listing fields that are already suitable for their current access patterns. After the pipeline implementation, the design prioritizes read performance and overwrite simplicity, since the workload is mainly read-heavy. This is reinforced by the new compound index on *listing_id* and date, which supports efficient retrieval of reviews for a given listing in descending date order.

3.2 Schema Implementation

Pipeline 1: Review Extraction and Migration Pipeline

```
db.listingsAndReviews.aggregate([
    { $unwind: "$reviews" },
    { $project: {
        _id: "$reviews._id",
        listing_id: "$reviews.listing_id",
        date: "$reviews.date",
        reviewer_id: "$reviews.reviewer_id",
        reviewer_name: "$reviews.reviewer_name",
        comments: "$reviews.comments"
    } },
    { $merge: {
        into: "reviews",
        on: "_id",
        whenMatched: "replace",
        whenNotMatched: "insert"
    } }
])
```

The first pipeline extracts the full embedded review history into a separate reviews collection. *$unwind* is used to deconstruct the reviews array so that each review becomes an individual pipeline document, *$project* reshapes each review into a clean, standalone schema, and *$merge* writes those documents into the new reviews collection. This stage is the core of the Reference Pattern because it separates the many-side review data from the parent listing document while preserving the link through *listing_id*. MongoDB documents *$unwind* as the stage used to deconstruct array fields and $merge as the stage used to write aggregation results into a target collection [3].

Pipeline 2: Parent Listing Restructuring Pipeline

```
db.listingsAndReviews.aggregate([
    { $addFields: {
        schema_version: 2,
        recentReviews: { $slice: ["$reviews", 3] }
    } },
    { $unset: "reviews" },
    { $merge: {
        into: "listingsAndReviews",
        on: "_id",
        whenMatched: "replace",
        whenNotMatched: "insert"
    } }
])
```

The second pipeline restructures the parent *listingsAndReviews* collection into the new hybrid schema. *$addFields* adds *schema_version*: 2, so the migration is explicit and creates a lightweight *recentReviews* array using *$slice* to retain only a small subset of embedded reviews. The number of subset reviews is limited to 3, which prevents this array from growing when the collection scales in the future. $unset then removes the original unbounded reviews array, and *$merge* writes the updated documents back into the parent collection.

This preserves quick access to a few recent reviews while preventing the full review history from continuously expanding the main listing document.

Pipeline 3: Listing–Review Relationship Validation Pipeline

```
db.listingsAndReviews.aggregate([
    { $match: { _id: "10006546" } },
    { $lookup: {
        from: "reviews",
        localField: "_id",
        foreignField: "listing_id",
        as: "reconstructedReviews"
    } },
    { $project: {
        name: 1,
        recentReviews: { $slice: ["$recentReviews", 3] },
        reconstructedReviews: { $slice: ["$reconstructedReviews", 3] }
    } }
])
```

The third pipeline is used to validate that the redesigned schema still preserves the logical relationship between a listing and its reviews. *$match* first selects one parent listing document by *_id*, and *$lookup* then joins that document to the new reviews collection by matching *listingsAndReviews*. *_id* with *reviews.listing_id*. This allows the review data to be reconstructed on demand, even though the full review history is no longer embedded inside the listing document. Finally, *$project* keeps the output concise by showing only the listing name, a limited sample from the stored *recentReviews* subset, and a limited sample from the joined *reconstructedReviews* array.


```
{
  _id: '10006546',
  name: 'Ribeira Charming Duplex',
  recentReviews: [
    {
      _id: '58663741',
      date: 2016-01-03T05:00:00.000Z,
      listing_id: '10006546',
      reviewer_id: '51483096',
      reviewer_name: 'Cátia',
      comments: '.......'
    },
    {
      _id: '62413197',
      date: 2016-02-14T05:00:00.000Z,
      listing_id: '10006546',
      reviewer_id: '40031996',
      reviewer_name: 'Théo',
      comments: "....."
    },
    {
      _id: '68310569',
      date: 2016-04-04T04:00:00.000Z,
      listing_id: '10006546',
      reviewer_id: '53859850',
      reviewer_name: 'Bart',
      comments: '.....'
    }
  ],
  reconstructedReviews: [
    {
      _id: '403055315',
      listing_id: '10006546',
      date: 2019-01-20T05:00:00.000Z,
      reviewer_id: '15138940',
      reviewer_name: 'Milo',
      comments: "....."
    },
    {
      _id: '364728730',
      listing_id: '10006546',
      date: 2018-12-31T05:00:00.000Z,
      reviewer_id: '91827533',
      reviewer_name: 'Mr',
      comments: '....'
    },
    {
      _id: '362865132',
      listing_id: '10006546',
      date: 2018-12-27T05:00:00.000Z,
```

```
51.       reviewer_id: '208880077',
52.       reviewer_name: 'Thomas',
53.       comments: '...'
54.     }]
55. }
56.
```

The result demonstrates three things at once (The comments are hidden with "…" to make the presentation clean). First, the parent listing still exists in the redesigned schema. Second, the full review history has been successfully moved out into a separate reviews collection. Third, the relationship has been preserved because the reviews can still be joined back to the listing whenever needed. *recentReviews* is the small, embedded subset kept inside the parent listing document after migration. These three reviews were preserved as a lightweight preview so the listing can still show some review information without loading the full review history every time.

```
1. db.reviews.createIndex({ listing_id: 1, date: -1 })
```

Finally, I indexed *listing_id* to support filtering by parent listing, while date: -1 supports sorting those reviews, the newest review comes first, and the date is descending. This makes the new schema more practical and optimized for real read operations particular operations defined by the CRUD table, by reducing the cost of fetching related reviews efficiently.

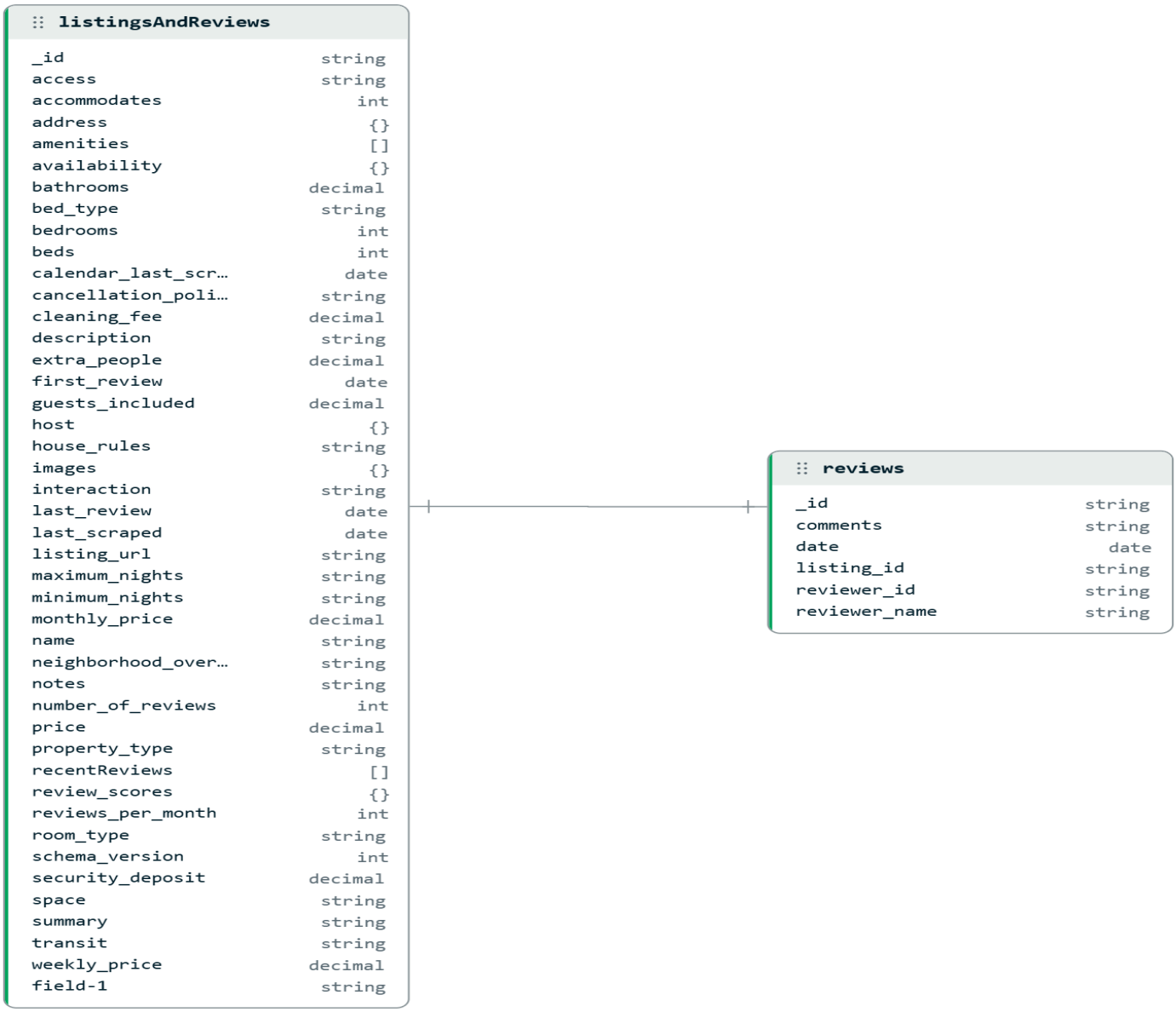


Figure 3 Schema Modelling exported from MongoDB Atlas

Figure 3 shows the redesigned schema after separating the unbounded review history from the parent listing document. *listingsAndReviews* stores the main listing attributes and a small *recentReviews* subset for fast detail-page access, while the full review history is held in the separate reviews collection. Each review document is linked to its parent listing through *listing_id*, creating a one-to-many relationship. This hybrid design improves scalability by preventing uncontrolled growth of the parent document while still supporting efficient retrieval of both listing-level data and related reviews.

4 References

[1] R. W. Saaty, "The analytic hierarchy process—what it is and how it is used," *Mathematical Modelling,* vol. 9, no. 3-5, pp. 161-176, 1987.

[2] K. D. Goepel, "AHP Priority Calculator," BPMSG, 26 Feb 2022. [Online]. Available: https://www.bpmsg.com/ahp/ahp-calc.php. [Accessed 7 April 2026].

[3] MongoDB Documentation Project, "MongoDB Documentation," 29 June 2015. [Online]. Available: https://web.cs.wpi.edu/~cs585/s17/Books/Books-PDF/MongoDB-manual.pdf. [Accessed 29 March 2026].